CSE470: Software Engineering

Software Requirements Specification (SRS)

End-of-Semester Submission Template

[Fill in every field below. Replace all bracketed instructions with your own content before submission. Delete this instructions line and all other bracketed instruction lines before you submit.]

ThesisFlow

Prepared By

Student ID	Name
23301601	Suhala Nagari
23301593	Rida Nawar
23201460	MD Mashrafi Mahmud Rafi
22241073	Himadri Shikhor Mondal

Project Information

Section	11
Section Faculty	Nazmus Sakib Touhid
GitHub Repository Link	https://github.com/ironMan-blip/ThesisFlow
Tech Stack Used	Python, FastAPI, PostgreSQL, React

1. Introduction

1.1 Purpose

This document describes the requirements of ThesisFlow, a web application for managing university thesis work. It lists what the system should do, who uses it, and how we plan to build it.

1.2 Scope

ThesisFlow handles a thesis from start to end for the CSE department. Students form groups, pick supervisors, upload proposals and reports, track experiments and expenses, and get reminders. Supervisors accept groups, review proposals, log meetings, and give marks. Panel members mark the final defense. The coordinator sets dates, supervisor limits, rubrics, and publishes grades. The thesis has three phases: P1, P2, and P3.

2. Overall Description

2.1 Product Perspective

ThesisFlow is a web application that handles the thesis process of the CSE department. Today the thesis process runs on email, Google Forms and spreadsheets. Our app puts all of it in one place. It uses a few outside services (Groq, OpenAlex, Semantic Scholar, arXiv, Brevo and Google Calendar), but the app still works if any of them is down.

2.2 Product High-Level Summary

Semester and archive: The coordinator sets up the semester and is responsible for opening and closing a semester. Old theses can be searched in the archives from the previous semesters.

AI help: An originality check of new proposals against published research worldwide and our archive, weekly alerts when a new paper clashes with a group's topic, and a chatbot that answers questions about the thesis rules.

Groups and supervisors: Students form groups, find teammates and request supervisors in order of maximum 3 choices. Supervisors limit how many groups they will supervise in a semester .

Proposals and deadlines: Groups upload different proposals and each proposal is stored as a different version. Each phase has a due date and a late penalty. Supervisors review proposals and groups can ask for more time if needed.

Notifications: In-app alerts and emails for every event, with reminders before deadlines.

Meetings and grading: Progress logs (meetings and weekly work), experiment and contribution tracking, expense management, marking rubrics, defense scheduling with no clashes, and final grades from the panel.

2.3 Users and Characteristics

Student: Forms or joins a group, submits work, logs weekly work, experiments and expenses, and asks the chatbot questions.

Supervisor: A faculty member who accepts groups, reviews proposals, holds meetings, approves group expenses, and gives marks.

Panel Member: A faculty member who marks the final defense. One faculty can be both supervisor and panel member.

Coordinator (Admin): Runs the semester, sets dates, supervisor limits, budget caps and rubrics, places leftover groups, and publishes grades.

2.4 Operating Environment

ThesisFlow is a web app. It runs in any modern browser on a desktop or a phone. For the demo, the whole app runs on a laptop with one command (*docker compose up*). No hosting is used.

2.5 Design and Implementation Constraints

This project follows some constraints including mandatory MVC architecture.

(a) Tech stack: Python, FastAPI, PostgreSQL 16 (with pgvector), React with Tailwind CSS.

(b) MVC: Model is in *backend/app/models/* (SQLAlchemy tables) and *backend/app/services/* (business rules). View is in *frontend/src/pages/* and *frontend/src/components/* (React). Controller is in *backend/app/controllers/* (FastAPI routers). Controllers only check the user and call a service. They do not hold business rules.

(c) AI features: All run inside our FastAPI backend. Smart search and the originality check use a local sentence-transformers model with pgvector. The guidelines chatbot uses the Groq API, with a local Ollama model as backup.

(d) Exception: Need to ask faculty if fastapi can be used as main backend language

2.6 Assumptions and Dependencies (Optional)

Backup apis are considered if an api feature fails for some reason

3. System Architecture

3.1 Architectural Overview

[Insert or describe an MVC architecture diagram showing how the Model, View, and Controller layers are organized in your project, and how they communicate.]

3.2 Technology Stack

Frontend: React, React Router, Tailwind CSS

Backend: Python 3.12, FastAPI, SQLAlchemy, Alembic, Pydantic

Database: PostgreSQL 16 with pgvector

AI: sentence-transformers (local), Groq API, Ollama (backup)

Outside services: OpenAlex, Semantic Scholar, arXiv (research papers), Brevo (email), Google Calendar

Hosting / Deployment: No hosting. Runs locally with Docker Compose

3.3 Class Diagram

[Draw the Class Diagram of your project showing all the classes and relationships necessary to implement your system]

4. Requirements and Features

List every requirement and its associated feature(s) below. Per the Project Outline (Section 3.2.1), all CRUD operations on the same entity by the same actor using the same logic count as ONE feature — do not split them. A minimum of 20 features is required.

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status

R1	Coordinator can run the semester	F1: Create the semester and open or close each step (form group, pick supervisor, P1, P2, P3, defense)	Coordinator	Model: Semester, Phase; View: SemesterSetupPage; Controller: SemesterController	Sprint 1	Planned
R2	Users can search old theses	F2: Upload past theses and search them by meaning, not only exact words	All users	Model: ArchivedThesis, DocChunk; View: ArchivePage; Controller: ArchiveController	Sprint 1	Planned
R3	New proposals are checked against all published research	F3: Compare a proposal with published research worldwide (OpenAlex, Semantic Scholar, arXiv), our thesis archive and other groups. Flag close matches with links to the papers. A person makes the final call	Supervisor, Coordinator	Model: OriginalityReport, ExternalPaper; View: OriginalityPage; Controller: OriginalityController	Sprint 2	Planned
R4	Users can ask about thesis rules	F4: Chatbot (Groq) answers from the official guidelines and shows the source	All users	Model: DocChunk; View: ChatbotPage; Controller: AssistantController	Sprint 2	Planned
R5	Groups falling behind are flagged	F5: A 0 to 100 health score for each group, with a warning when it drops	Supervisor, Coordinator	Model: HealthScore; View: GroupHealthPage; Controller: HealthController	Sprint 3	Planned
R6	Students can form groups and find teammates	F6: Create a group of 3 to 5, invite members, accept or decline. Invites expire in 3 days. Students without a group search others by interests and skills and invite them	Student	Model: Group, GroupMember, Invitation, StudentProfile; View: GroupPage, TeammateFinderPage; Controller: GroupController	Sprint 1	Planned

R7	Groups can manage thesis expenses	F7: Group logs thesis costs (GPU or cloud time, datasets, hardware, printing) with receipts. The coordinator sets a	Student, Supervisor, Coordinator	Model: Expense, Budget; View: ExpensePage; Controller: ExpenseController	Sprint 1	Planned

		budget cap and the group is warned near it. The supervisor approves or rejects each expense. Export a cost summary				
R8	Groups can request supervisors	F8: Group picks 3 supervisors in order. If #1 says no or does not reply, it moves to #2, then #3. Coordinator sets a max number of groups per supervisor and full supervisors are skipped. Groups left without a supervisor wait in a queue and the coordinator places them	Student, Supervisor, Coordinator	Model: SupervisorRequest, FacultyProfile, Waitlist, AllocationLog; View: SupervisorRequestsPage, AllocationPage; Controller: SupervisorRequestController	Sprint 2	Planned
R9	Groups are alerted when new research clashes with their topic	F9: Every week the app runs the F3 search again on newly published papers. If a new paper is close to a group's topic, the group and supervisor get an alert. Shows a history of alerts with similarity scores	Student, Supervisor	Model: NoveltyAlert, OriginalityReport; View: NoveltyAlertsPage; Controller: NoveltyController	Sprint 3	Planned
R10	Groups get supervisor suggestions	F10: Suggest supervisors whose research papers (OpenAlex) match the group topic. Only supervisors with free slots (F8) are shown	Student, Coordinator	Model: FacultyProfile; View: SupervisorSuggestPage; Controller: RecommendationController	Sprint 2	Planned
R11	Groups can upload proposals	F11: Upload proposal PDF with details. Every upload is a new version, old ones are kept. View and download versions	Student	Model: Proposal, ProposalVersion; View: ProposalPage; Controller: ProposalController	Sprint 1	Planned

R12	Each phase has a deadline	F12: Set P1, P2, P3 due dates, grace time and late penalty. App counts late days and penalties. Timeline with countdowns	Coordinator, Student	Model: MilestoneTemplate, GroupMilestone, Deliverable; View: DeadlinePage; Controller: MilestoneController	Sprint 1	Planned
R13	Supervisors can review proposals	F13: Comment by section, students reply. Approve, ask for changes, or reject	Supervisor, Student	Model: FeedbackThread, FeedbackComment; View: ReviewPage; Controller: ReviewController	Sprint 2	Planned
R14	Users get notified	F14: Bell icon in the app plus email (Brevo). Reminders 3 days and 1 day before each deadline. Email settings	All users	Model: Notification, EmailOutbox; View: NotificationBell; Controller: NotificationController	Sprint 2	Planned
R15	Groups can ask for more time	F15: Group asks for an extension, supervisor gives an opinion, coordinator decides	Student, Supervisor, Coordinator	Model: ExtensionRequest; View: ExtensionPage; Controller: ExtensionController	Sprint 3	Planned

R16	Supervisors and students log progress	F16: Record meetings, who came, notes and tasks for each member. Each member also logs weekly hours and work done. Flags members quiet for 2 weeks	Supervisor, Student	Model: Meeting, MeetingAttendance, ActionItem, ContributionLog; View: MeetingPage, WorkLogPage; Controller: ProgressController	Sprint 1	Planned
R17	Groups track experiments and member contributions	F17: Group adds experiments with dataset, setup, owner and status (planned, running, done, failed) and records results. Shows each member's share of the experiment	Student, Supervisor	Model: Experiment, ExperimentRun; View: ExperimentPage; Controller: ExperimentController	Sprint 1	Planned

		work, which feeds the health score (F5)				
R18	Marking uses rubrics	F18: Make a rubric for P1, P2, P3 with criteria and weights that add up to 100%	Coordinator	Model: Rubric, Criterion; View: RubricPage; Controller: RubricController	Sprint 2	Planned
R19	Defenses are scheduled	F19: Make defense slots with no teacher or room clash. Send Google Calendar invites	Coordinator	Model: DefenseSlot, PanelAssignment; View: DefenseSchedulePage; Controller: DefenseController	Sprint 2	Planned
R20	Final grades are published	F20: Combine panel marks. If marks differ by more than 15%, coordinator checks. Then grades are published	Panel Member, Coordinator	Model: Evaluation, Score, GradeRecord; View: GradesPage; Controller: GradeController	Sprint 3	Planned

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20
Total features listed in Section 4 above	20
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	0 (work in progress)

5. External Interface Requirements (Optional)

Interface Type	**Description**
User Interface	Web pages made with React. Each role sees its own menu. Every page shows loading, empty, error and success states. Works on desktop and phone screens.
Hardware Interface	No special hardware. A computer or phone with a browser and internet.
Software Interface (APIs, third-party services)	Groq (chatbot), OpenAlex, Semantic Scholar and arXiv (research papers), Brevo (email), Google Calendar (defense invites). All free apis, each with a backup.

Communication Interface	The React frontend talks to the FastAPI backend through a REST API using JSON over HTTP. Login uses JWT tokens. Emails go out through Brevo.

6. Non-Functional Requirements (Optional)

Category	**Requirement**
Performance	Pages should load in a few seconds. Slow work (AI search, emails) runs in the background.
Security	Passwords are hashed. The backend checks the role and owner on every request. Only real PDFs up to 20 MB can be uploaded. API keys stay in a .env file.
Usability	Simple pages with clear messages. All times shown in Dhaka time.
Reliability / Availability	If an outside service is down, the app still works using its backup. The same email is never sent twice.
Scalability	Built for one department. Up to a few hundred users in one semester.

7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

Sprint	**Goals Set (Start of Week)**	**Attainment Reported (End of Week)**	**Features Completed This Sprint**
Sprint 1 (24–30 Oct)	Basic features. Coordinator opens the steps, students make groups and find teammates, groups log progress, experiments and expenses, old theses are searchable, proposal upload and P1/P2/P3 deadlines.	To be updated at the review on Fri 30 Oct	Target: F1, F2, F6, F7, F11, F12, F16, F17
Sprint 2 (31 Oct–6 Nov)	Full flow from group to approved proposal with emails. Originality check against published research. Chatbot with sources. Defense slots in Google Calendar.	To be updated at the review on Fri 6 Nov	Target: F3, F4, F8, F10, F13, F14, F18, F19

Sprint 3 (7–13 Nov)	Last feature for each member, including weekly novelty-clash alerts. Test each other's features, fix bugs, tag version 1.0.	To be updated at the review on Fri 13 Nov	Target: F5, F9, F15, F20
Sprint 4 (14–20 Nov)	No new features. Fix bugs, record the video, finish the SRS, practise for the viva, submit.	To be updated on Fri 20 Nov	None (wrap-up week)

7.1 Deviations and Lessons Learned

Changes after the faculty SRS review (made in Sprint 1): R3 now checks proposals against all published research, not only our own archive. R6 and R7 were merged into F6 (group formation). R9 (supervisor limit) became part of F8 and F10 instead of a separate feature. R16 and R17 were merged into F16 (progress log). Three new features took the freed places: F7 expense management, F9 novelty-clash alerts and F17 experiment and contribution tracking. F10 moved to Sprint 2 and F9 to Sprint 3. This will be updated after each Friday review. Our rule: if a feature is late, it is finished on Monday or Tuesday of the next week, and the next feature starts on Wednesday. We will say so at the review.

8. Testing Summary

Unit tests (pytest): Business rules like late penalty, weighted marks, health score, supervisor limits, budget caps and similarity thresholds.

API tests (pytest): Every endpoint, for both right and wrong use. A user cannot open another user's data. Postman will also be used to test different api routes.

Deadline tests: A fixed clock to test cases like "late by 1 minute" and the grace time.

Outside service tests: Groq, OpenAlex, Semantic Scholar, arXiv, Brevo and Google Calendar are faked, so we can test timeouts and errors without calling them.

Frontend tests (Vitest): Forms and page states.

Manual tests: A full thesis run with demo data before every Friday review. In Sprint 3 each member tests another member's 5 features.

Tests run on every pull request with GitHub Actions.

9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)

[By submitting this document, you confirm that you are able to personally explain, modify, and debug every piece of code listed above during your viva.]

10. Individual Contribution Summary

[Since each student develops their project individually, summarize your own work: which features you implemented, approximate time spent per sprint, and any challenges you solved independently. If you collaborated with your 4-student group on non-code matters (e.g. idea discussion), state that clearly and distinguish it from your individual implementation work.]

11. Pre-Submission Checklist

Checklist Item	**Confirmed (Yes/No)**
The application implements the MVC architecture	
At least 20 features are listed and each is correctly counted per the granularity rule	
No forbidden framework (e.g. Django, Flask as primary backend) was used	
All four sprints' goals and attainment are recorded in Section 7	
The GitHub repository is up to date and the section faculty is added as a collaborator	
The AI Usage Disclosure table (Section 10) is complete and honest	
I can personally explain and defend every feature listed in this document during the viva	